USER GUIDE

PrimeOffice

Pixel office for your AI coding agents

PrimeOffice is a **pixel office for your AI coding agents**. Every Claude Code session on this PC (CLI, Desktop app or IDE) that was active recently becomes a character: it walks to a desk, **types** while it edits files or runs commands, **reads** while it searches, and raises a **bubble** when it needs you. Around the office: **Projects** (check a project folder against the standard and list everything missing), **History** (the latest sessions: last prompt, what was just done, last reply) and **Agents** (which agents are on this PC).

Version: 0.1.1 · **Generated:** 2026-11-09

The same content as the in-app Help (press F1).

Contents

1 **Getting started**
What PrimeOffice is, how it finds agent sessions, what it reads and writes, the layout, dropping folders, shortcuts, appearance and updates.

2 **Office — the pixel office**
Every active agent session is a character: it types while editing, reads while searching, raises a bubble when it needs you. Filter by project and time, pick the crew, click a character for details.

3 **Office — layout editor**
The office's Layout button: place furniture, paint floors, walls and carpets, draw folder areas, erase tiles; Undo / Redo / Save / Reset. The layout saves itself to layout.json.

4 **Agent details**
The details of one session (Office and History): status, model, context, last prompt, what is running, recent actions, last reply.

5 **Projects — project check**
Open a folder, check it against the project standard (agent sessions, CLAUDE.md, git, docs/SPEC.md, docs/PROGRESS.md…) and see how to fix every missing item.

6 **History — recent sessions**
The latest sessions of every project or one: title, status, last prompt, last three actions; click one for the full view.

7 **Agents — providers on this PC**
Claude Code, Codex CLI (ChatGPT), Gemini CLI and DeepSeek: found on this PC or not, where their sessions are, whether the office can show them.

8 **Command line**
primeoffice.exe check / history / providers: check a project, read its history, detect agents from a terminal or CI, with a JSON report and exit codes.

1 Getting started

What PrimeOffice is, how it finds agent sessions, what it reads and writes, the layout, dropping folders, shortcuts, appearance and updates.

What is PrimeOffice?

PrimeOffice is a **pixel office for your AI coding agents**. Every Claude Code session on this PC (CLI, Desktop app or IDE) that was active recently becomes a character: it walks to a desk, **types** while it edits files or runs commands, **reads** while it searches, and raises a **bubble** when it needs you. Around the office: **Projects** (check a project folder against the standard and list everything missing), **History** (the latest sessions: last prompt, what was just done, last reply) and **Agents** (which agents are on this PC).

- The app's interface is in English; this guide shows button and option names in **bold** exactly as on screen.
- Open this guide any time with `F1` (it opens on the tab you are in, even while the office has focus), the **?** button in the header, or Ctrl+K → **Help / Hướng dẫn**.

How PrimeOffice finds sessions

Claude Code writes each session as a transcript `<session>.jsonl` in `%USERPROFILE%\.claude\projects\<folder>\`, where `<folder>` is the session's working directory with every character other than letters, digits and `-` turned into `-` (for example `D:\Game` → `D--Game`). PrimeOffice only **reads** these files.

- A background observer checks **every second** for transcripts that changed within the **Active in** window (30 minutes by default) and reads only the new lines. When a session stays quiet longer than that, its character leaves the office.
- No hooks, no change to `~/.claude/settings.json`, it never launches or steers an agent, and nothing is sent over the network.
- Codex (`~/.codex/sessions`) and Gemini (`~/.gemini/tmp`) are only checked for existence today — they do not join the office yet (see the **Agents** topic).
- **Waiting for permission** is a **guess** from the transcript: a tool that has not finished and has been quiet for over 10 seconds. Long-running commands (Bash, PowerShell), subtasks and web lookups are never guessed as waiting.

What PrimeOffice reads and writes

- **Reads**: Claude Code transcripts (above); in a project folder it only checks **which files exist** (CLAUDE.md, .git, docs/SPEC.md…), without reading them.
- It **never writes** into a project folder or into an agent's own files.
- **Writes** to `%LOCALAPPDATA%\com.phineas.primeoffice\`: `projects.json` (the projects you opened), `layout.json` (the office layout saved by the editor) and `office.json` (the office's Settings, agents' seats, folder ↔ area mapping).
- UI preferences (office filter, crew, appearance, help language) are remembered in the app window's storage. The command line's `--report` writes a JSON file exactly where you tell it.

Layout

- **Navigation rail (left)**: the 4 tabs, in the order of Ctrl+1 … Ctrl+4 — **Office**, **Projects**, **History**, **Agents**. At the bottom: **Settings** (Ctrl+,), **About & contact**, **Command palette** (Ctrl+K) and **Show log / Hide log** (with an error count while the log is closed).
- **Header**: `PrimeOffice / <tab>`, the **Search actions…** box (opens the command palette), the **?** button (this guide), the version pill (`browser preview` outside the desktop app — no real data is read there) and `via Abyss` when Abyss launched it.
- **Log (bottom)**: messages from the UI and the Rust side (projects checked, layout saved, errors…). Toggle it with Ctrl+`; Ctrl+K → **Clear log** empties it.

Opening project folders

Drag & drop a folder anywhere on the window, or use **Open folder…** / Ctrl+O / Ctrl+K → **Open project folder…**. The folder is added to the **Projects** list and checked, then, depending on the tab you are in:

- **Office**: if the project is **Ready**, the office filters to it; if not, you are taken to **Projects** to see what is missing.
- **Projects**: the project is selected and its report shown.
- **History**: History filters to that project.
- **Agents**: you are taken to **Projects** to see that project's report.
- Only the first item is used when you drop several. Dropping a file (not a folder) fails the **Folder exists and can be read** check.

Shortcuts

`F1`	Open / close this guide (on the current tab's topic)
`Ctrl` `K`	Command palette: tabs, open a folder, run again, updates, log, settings…
`Ctrl` `1` … `Ctrl` `4`	Switch tab (Office, Projects, History, Agents)
`Ctrl` `O`	Open a project folder (as if dropped on the current tab)
`F5`	Run the current tab again: Office refreshes the agent list, Projects re-checks the selected project, History reloads, Agents detects again. Never writes anything
`Ctrl` `` ` ``	Show / hide the log
`Ctrl` `,`	Settings (appearance)
`Esc`	Close a dialog / the command palette / this guide

- Right after you click in the office, keys go to the office: only `F1` is passed on. Click the header or the list on the right before using Ctrl+K, Ctrl+1…4 or F5. The layout editor's keys are in the **Office — layout editor** topic.

Appearance and updates

- **Settings** (Ctrl+,): light / dark / Windows theme, UI and code fonts, UI scale, effects level. It does not change the pixel office (the office has its own **Settings**, see the **Office** topic).
- Ctrl+K → **Check for updates…** (or in **About & contact**). Release builds also check once at startup. When a new version exists, the header shows **Update to vX.Y.Z** to download, install and restart.
- When Abyss launches PrimeOffice, it does not check by itself: Abyss shows the update and installs it when the app closes.

2 Office — the pixel office

Every active agent session is a character: it types while editing, reads while searching, raises a bubble when it needs you. Filter by project and time, pick the crew, click a character for details.

Quick start

1. Use Claude Code as usual (terminal, Desktop app or IDE). Within a second of its transcript changing, the session's character walks into the office.
2. Read the label above the character: what it is doing (`Editing App.tsx`, `Running: npm test`, `Searching code`…) and the folder name.
3. Click a character (or a row in the list on the right) to see its last prompt, recent actions and last reply — see the **Agent details** topic.

Top toolbar

- **Project**: **All projects** or one project. Only projects that are **Ready** in the Projects tab are listed.
- **Open folder…**: opens a project folder; if it is Ready the office filters to it, otherwise you go to Projects.
- **Active in**: `15 min`, `30 min`, `1 h`, `4 h`, `24 h` — sessions whose transcript changed within this window are in the office; quieter ones leave. The same window decides the **Idle** status in History.
- **Crew**: **PrimeOffice crew** (8 characters drawn by the app's own generator) or **Classic (pixel-agents)**. Switching crew reloads the office.
- The **N agents** pill counts the characters; the arrow button (**Agent list**) hides / shows the right column. Project, Active in and Crew are remembered for next time.

Characters, labels and bubbles

- **Typing**: editing / writing files, running commands, other tools. **Reading**: `Read`, `Grep`, `Glob`, `WebFetch`, `WebSearch`.
- **Green check**: the turn is done; the agent waits for your next prompt.
- **Waiting for input** label: the agent asked you something (`AskUserQuestion`) or waits for plan approval (`ExitPlanMode`).
- **…** bubble: most likely waiting for your permission (a guess: a tool quiet for more than 10 seconds). Check the Claude Code window to be sure.
- The small bar under the label is the **context** used; hover it for the token count.
- Subtasks (`Task` / `Agent`) show as a `Subtask: …` label on the agent itself, not as separate characters yet.

Moving around the office

- Click a character to select it: the camera follows it and its details open on the right. Click it again or click empty floor to deselect.
- With a character selected, click a **free seat** to move it there; click its own seat to send it back. Seats are remembered per session, across crew switches and app restarts.

- Mouse wheel / two-finger scroll: pan. Ctrl+wheel or the **+ / −** buttons at the top left: zoom. Middle-button drag: pan.
- The column of small cards on the office's right edge: one card per agent (portrait + status dot); click to select and follow.
- The **×** (**Close agent**) button sends the character out of the office but **does not stop** the agent (PrimeOffice only observes): it walks back in, to the same seat, as soon as its session does something new. A character also leaves by itself once its session is quiet for longer than **Active in**.

The office's bottom bar

- **Layout**: turns on the layout editor — see the **Office — layout editor** topic.
- **Settings** (the office's own): **Sound Notifications** (a chime when an agent finishes / needs you), **Always Show Labels** (labels above every character), **Show Areas** (only when Areas are available), **Debug View** (a raw status list instead of the office).
- **Sound Notifications**, **Always Show Labels** and **Show Areas** are saved (`office.json`) and survive crew switches and app restarts. Defaults: sound off, labels always shown, areas hidden.

The right column

- Each row: the session title (or folder name), status, what it is doing or did last, folder · time. The list refreshes every 2 seconds; F5 refreshes it at once.
- Statuses: **Working** (running), **Needs you** (asking / waiting for permission), **Turn done** (turn finished), **Idle** (quiet longer than Active in).
- Click a row for its details; **Back to the list** returns.

Empty office?

- **The office is quiet** means no session changed within **Active in** (and in the selected project). Try a longer window or **All projects**.
- Open the **Agents** tab: Claude Code should be **Active** or **Found**. **Not found** means `~/.claude/projects` does not exist yet — run `claude` once.
- The project filter matches the session's exact working directory: sessions started in a parent or a subfolder do not count.

3 Office — layout editor

The office's Layout button: place furniture, paint floors, walls and carpets, draw folder areas, erase tiles; Undo / Redo / Save / Reset. The layout saves itself to layout.json.

Turning the editor on / off

1. Click **Layout** in the office's bottom bar (it lights up while editing).
2. Pick a tool in the button row: **Furniture**, **Floor**, **Walls**, **Pets**, **Areas** (when available), **Erase**. Click the active tool again to go back to selecting.
3. Click **Layout** again (or press `Esc` a few times) to leave.

Tools

- **Furniture**: pick a group (**Desks**, **Chairs**, **Storage**, **Tech**, **Decor**, **Wall**, **Misc**), pick an item, click the floor to place it. **Copy** takes the type of an item already placed. The colour for new items is set in the colour panel above.
- Select a placed item (click it): drag to move it (hold Alt as you start dragging to drag out a copy), use the rotate / delete buttons next to it, and the colour panel to recolour it (**Reset** restores the original).
- **Carpet** (inside Furniture): paint carpets; pick a carpet style, colour and **Accent** colour; **Copy** (`P`) takes style + colours from an existing tile.
- **Floor**: pick a floor pattern, then click / drag to paint; **Color** opens colour and **Contrast**; **Copy** takes pattern + colour from an existing tile.
- **Walls**: click / drag to add or remove walls; pick a wall set and colour.
- **Erase**: turns tiles into void (outside the room). In Floor, Walls, Carpet, Areas and Erase, the **right mouse button** erases too.
- **Pets**: PrimeOffice ships no pets yet, so this panel is empty.

Folder areas (Areas)

Areas reserve part of the office for a folder: that folder's agents sit inside it. The **Areas** button only shows when the layout already has areas or the office has seen at least one agent (its folder can be mapped).

1. Type a name in **Area name...**, pick a colour, click **Add**.
2. Select the area's card, then paint its tiles on the map (right-click removes).
3. In the card, **Map a folder...** assigns a folder (an agent's folder name); double-click the name to rename it, use the remove button to delete the area.

- Areas are saved with the layout; the folder ↔ area mapping is saved on its own (`office.json`), so both survive crew switches and app restarts.

Saving, Undo, Reset

- Every change is **saved automatically** about half a second later to `%LOCALAPPDATA%\com.phineas.primeoffice\layout.json` (the log says `Layout saved`). Without that file the office uses the bundled default layout.
- Once something changed, the top bar shows **Undo** (Ctrl+Z), **Redo** (Ctrl+Y / Ctrl+Shift+Z), **Save** and **Reset**.
- **Save** marks the current layout as the checkpoint; **Reset** → **Yes** returns to it (the last Save, or the layout the office opened with).
- PrimeOffice has no layout export / import yet; copy `layout.json` to back it up.

Editor keys

`R`	Rotate the selected item
`T`	Toggle the selected item's state (e.g. on / off)
`Delete` / `Backspace`	Delete the selected item
`Ctrl` `Z`	Undo
`Ctrl` `Y` / `Ctrl` `Shift` `Z`	Redo
`P`	In Carpet: copy a carpet from an existing tile
`Esc`	Steps back: drop the item in hand → close the tool → deselect → leave the editor

- These keys only work while the office has focus (click in the office first).

4 Agent details

The details of one session (Office and History): status, model, context, last prompt, what is running, recent actions, last reply.

Where it opens

- **Office**: click a character or a row in the right column (last 15 actions).
- **History**: pick a session in the left list (last 50 actions).

Reading the panel

- **Title**: the title set in Claude Code (custom title / agent name) or its summary; otherwise the start of the first prompt; `Untitled session` when empty.
- Info line: agent (`Claude Code`), where it runs (`cli` , `Desktop app` ...), model, last activity, prompt count; below it, the working folder.
- **Context N%**: context used (out of 200k tokens, or 1M for `[1m]` models); green below 60%, amber up to 85%, red above. Hover for the token count.
- **Last prompt**: your latest prompt (commands such as `/init` show as the command), up to 500 characters.
- **Running now**: tools that started and have not finished.
- **Recent actions**: newest first, with the time: reads, edits, commands, searches, web, subtasks, questions, planning, skills...
- **Last reply**: the agent's last text answer, up to 500 characters.

How the status is worked out

- **Working**: the turn is running.
- **Needs you**: it asked you / waits for plan approval, or a tool has been quiet for over 10 seconds (guessed as a permission prompt).
- **Turn done**: the turn ended, or the last record is a text reply and it has been quiet for over 5 seconds.
- **Idle**: the transcript has not changed within the Office's **Active in** window.

Tips

- You can select and copy the **Last prompt** / **Last reply** text.
- The summary is mechanical (from tools and prompts); no LLM is called.
- Very long transcripts are read head (256 KB) and tail (4 MB) only, so the prompt count can be partial.

5 Projects — project check

Open a folder, check it against the project standard (agent sessions, CLAUDE.md, git, docs/SPEC.md, docs/PROGRESS.md…) and see how to fix every missing item.

How to use it

1. Click **Open folder…** (or Ctrl+O, or drop a folder on the window).
2. The project goes to the top of the left list — green dot **Ready**, red dot **not ready** — and its report shows on the right.
3. Fix the failed items (each has a **Fix:** line with a copy button), then click the refresh button (**Check again**, F5).
4. Once it is **Ready**: **View in office** shows only this project's agents; **History** opens its history.

The report

- The **Ready** or **Not ready — N required missing** pill, the path, session counts per agent and the last one.
- The coloured banner lists what is missing at a glance; **Project standard — X/Y passed** lists every item, failed ones first, with its code (`agent.sessions`…).
- Levels: **required** (missing means it cannot be viewed yet), **recommended**, **optional**.
- The bin button **Remove from the list** only removes it from the list — the folder is not touched. The folder button shows it in Explorer.

Checks and how to fix them

- **Folder exists and can be read** (required, `folder.exists`): the folder was moved, renamed or is on a missing drive. Open it again and remove the old entry.
- **Agent sessions for this folder** (required, `agent.sessions`): no agent has run in **this exact** folder yet. Open a terminal there and run `claude` once (or open this folder in Claude Desktop / your IDE), then check again. Sessions started in a parent folder do not count.
- **Agent instructions (CLAUDE.md or AGENTS.md)** (recommended, `agent.instructions`): run `/init` in Claude Code in this folder, or write `CLAUDE.md` yourself. `AGENTS.md` and `.claude/CLAUDE.md` count too.
- **Git repository** (recommended, `vcs.git`): run `git init` in the folder.
- **Spec (docs/SPEC.md)** (recommended, `abyss.spec`): write `docs/SPEC.md` — goals, main flows, architecture (per Abyss `docs/app-specs/00_SHARED_FRAMEWORK.md` §3).
- **Progress log (docs/PROGRESS.md)** (recommended, `abyss.progress`): add `docs/PROGRESS.md` — status, phases, decisions (template in §15 of the shared framework).
- **Project agent settings (.claude/)** (optional, `agent.settings`): create `.claude/` if the project needs its own settings, skills or agents.
- **Changelog** (optional, `abyss.changelog`): add `docs/CHANGELOG.md` (or `CHANGELOG.md` at the root).
- **README.md** (optional, `abyss.readme`): add a `README.md` saying what the project is and how to run it.

Notes

- PrimeOffice **never creates** these files for you: it only checks whether they exist. The project list is kept in `%LOCALAPPDATA%\com.phineas.primeoffice\projects.json`.
- Every listed project is re-checked whenever the tab opens. The line under each name in the left list counts what is missing by level, e.g. `2 recommended, 1 optional missing` (or `nothing missing`); a project that is not Ready says `not ready`.
- The same check runs in CI: `primeoffice check <folder>` (see the **Command line** topic).

6 History — recent sessions

The latest sessions of every project or one: title, status, last prompt, last three actions; click one for the full view.

How to use it

1. Pick a **Project**: **All projects** (every Claude Code session on this PC) or a project opened in Projects.
2. Pick **Show**: `10`, `30`, `60` or `120 latest` — how many of the newest sessions (by when the transcript last changed).
3. Click a session on the left to see its details on the right (see the **Agent details** topic). **Refresh** (F5) reloads.

What each row shows

- The title (or `Untitled session`) and status (**Working**, **Needs you**, **Turn done**, **Idle**).
- The last prompt (2 lines), the three latest actions with an icon per kind, then folder · time.

Tips

- Drop a folder on the History tab to open it as a project and filter History to it at once.
- **Idle** uses the Office's **Active in** window: sessions older than that are Idle.
- Only Claude Code sessions appear here; Codex and Gemini come later. History only reads transcripts; it never deletes or edits anything.
- The Project / Show choice is not remembered when the app restarts.

7 Agents — providers on this PC

Claude Code, Codex CLI (ChatGPT), Gemini CLI and DeepSeek: found on this PC or not, where their sessions are, whether the office can show them.

What each card shows

- State: **Active** (sessions changed in the last 30 minutes), **Found** (its session folder exists), **Not found**, **Planned** (known but not supported yet).
- A note, the session folder path, recent sessions (30 minutes) and **Shown in the office** / **Not in the office yet**.
- **Refresh** (or F5) detects again.

Each agent

- **Claude Code**: fully supported — `~/.claude/projects`, CLI, Desktop app and IDE sessions. **Not found**: run `claude` once to create the folder.
- **Codex CLI (ChatGPT)**: only `~/.codex/sessions` is checked; reading its sessions comes in a later version.
- **Gemini CLI**: only `~/.gemini/tmp` is checked; reading its sessions comes in a later version.
- **DeepSeek**: always **Planned** — no local session format yet; it will come through the tool that runs DeepSeek.

- This tab only reads; it never installs, configures or starts an agent. Dropping a folder on this tab (or Ctrl+O) opens it as a project and takes you to **Projects**.

8 Command line

primeoffice.exe check / history / providers: check a project, read its history, detect agents from a terminal or CI, with a JSON report and exit codes.

Commands

`check <thư mục>`	Check against the project standard, print every missing item with its fix
`history <thư mục> --last N`	The N newest sessions (default 5): last prompt, recent actions, last reply
`providers`	Claude Code / Codex / Gemini / DeepSeek on this PC
`--help`	Help and examples

- `check` : `--report r.json` writes the JSON report (the same structure the app shows), `--fail-on warn|error` (default `error`), `--quiet` prints only the summary and errors. `history` : `--report r.json`.
- Exit codes: `0` done, no issue at the `--fail-on` level; `1` issues at that level (e.g. a project that is not Ready); `2` bad arguments or a missing folder.
- In `cmd.exe` use `start /wait primeoffice.exe …` to wait for the exit code; PowerShell and CI call it directly. Any other first argument opens the app window as usual.
- `primeoffice.exe <folder>` (or **Open with** PrimeOffice on a folder in Explorer): the window opens and the folder is opened as if dropped on the **Office** tab — the office filters to it when it is Ready, otherwise you are taken to **Projects**. An argument that is not a folder is ignored (with a warning in the log).